TAREA GRUPO 3

Autores:

- Calahorrano Geovanna.
- Vega Ivan.
- Yanez Carlos

Tarea Parte 1

1.-Cargar una imagen con OpenCV

2.-Mostrar la imagen cargada

```
# Descargar librerias
!pip install opencv-python
!pip install matplotlib
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.
```

```
# Importar bibliotecas
import cv2
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.image as mpimg


# DEFINIR RUTA
ruta_archivo = r"/content/imagen1.jpg"

# Cargar imagen
try:
    img = mpimg.imread(ruta_archivo)

    # Mostrar imagen
    plt.figure(figsize=(8,6))
    plt.imshow(img)
    plt.title("Imagen original")
    plt.axis("off")
    plt.show()

except FileNotFoundError:
    print("Error: no se encontró la imagen. Verifique la ruta.")
```

Imagen original

3.-Convertirla a Escala de grisis

```
# Leer imagen
```

```
# 'img' ya fue cargada en la celda anterior (e9166981) usando mpimg.imread.
# cv2.imread es más adecuado para operaciones de OpenCV.
# Para asegurar que cv2 tenga la imagen en su formato BGR, se puede recargar con cv2.imread
# o convertir la imagen cargada por mpimg a BGR si es necesario.
# Dado que la celda e9166981 ya cargó 'img' y 'imagePath' se actualizó,
# y 'mpimg.imread' carga en RGB, para cv2.cvtColor(BGR2GRAY) se necesita que 'img' esté en BGR.
# Si 'img' fue cargada por mpimg, su formato es RGB. cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) esperaría BGR.
# Vamos a cargarla de nuevo con cv2.imread para asegurar el formato correcto para OpenCV.

img_cv2 = cv2.imread(ruta_archivo)

if img_cv2 is None:
    print(f"Error: No se pudo cargar la imagen desde {ruta_archivo} con cv2.imread.")
else:
    # Convertir a escala de grises
    gray_image = cv2.cvtColor(img_cv2, cv2.COLOR_BGR2GRAY)

    # Mostrar imagen en escala de grises
    plt.figure(figsize=(8,6))
    plt.imshow(gray_image, cmap='gray')
    plt.title("Imagen en escala de grises")
    plt.axis("off")
    plt.show()
```

Imagen en escala de grises

4.-Aplicar el filtro de deteccion de bordes Canny

5.-Mostrar la imagen procesada

6.-Buscar otro filtro diferente a canny

7.-Aplicar el filtro a la imagen original y la procesada con el filtro canny

8.-Mostrar los resultados de las 2 imagenes procesadas con el nuevo filtro

```
# -------- FILTRO CANNY --------
canny_edges = cv2.Canny(gray_image, 100, 200)

# -------- NUEVO FILTRO: SOBEL --------
# Sobel en X
sobelx = cv2.Sobel(gray_image, cv2.CV_64F, 1, 0, ksize=3)

# Sobel en Y
sobely = cv2.Sobel(gray_image, cv2.CV_64F, 0, 1, ksize=3)

# Magnitud del gradiente
sobel_original = cv2.magnitude(sobelx, sobely)

# -------- SOBEL aplicado a la imagen con CANNY --------
sobelx_canny = cv2.Sobel(canny_edges, cv2.CV_64F, 1, 0, ksize=3)
sobely_canny = cv2.Sobel(canny_edges, cv2.CV_64F, 0, 1, ksize=3)

sobel_canny = cv2.magnitude(sobelx_canny, sobely_canny)

# -------- MOSTRAR RESULTADOS --------
```

```
# 3. Configurar la visualización


plt.figure(figsize=(18,6))

plt.subplot(1,3,1)
plt.title("Imagen con Canny")
plt.imshow(canny_edges, cmap='gray')
plt.axis('off')

plt.subplot(1,3,2)
plt.title("Sobel aplicado a imagen original")
plt.imshow(sobel_original, cmap='gray')
plt.axis('off')

plt.subplot(1,3,3)
plt.title("Sobel aplicado a imagen con Canny")
plt.imshow(sobel_canny, cmap='gray')
plt.axis('off')

plt.show()
```

Detección de rostros- método 1

```
# Importar las bibliotecas necesarias
import cv2
import matplotlib.pyplot as plt
```

```
# Definir la ruta de la imagen de entrada
imagePath = r"/content/imagen1.jpg"
```

```
# Mostrar la forma (shape) de la imagen
# Esto proporciona información sobre las dimensiones de la imagen
img.shape
```

```
(3407, 6937, 3)
```

```
# Convertir la imagen a escala de grises
gray_image = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Mostrar la forma (shape) de la imagen en escala de grises
gray_image.shape
```

```
(3407, 6937)
```

```
# Cargar un clasificador pre-entrenado para la detección de caras
# Descargar el archivo haarcascade si no está presente
!wget -nc https://raw.githubusercontent.com/opencv/opencv/4.x/data/haarcascades/haarcascade_frontalface_default.xml

face_classifier = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

# Detectar caras en la imagen en escala de grises
face = face_classifier.detectMultiScale(
    gray_image, scaleFactor=1.1, minNeighbors=5, minSize=(40, 40)
)
```

```
# Dibujar rectángulos alrededor de las caras detectadas
for (x, y, w, h) in face:
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 4)

# Convertir la imagen al formato RGB para mostrarla con Matplotlib
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Crear una figura de Matplotlib para mostrar la imagen
plt.figure(figsize=(20,10)) # Configuramos el tamaño de la figura
plt.imshow(img_rgb) # Mostramos la imagen en la figura
plt.axis('off') # No dibujamos los ejes de la figura
```

```
--2026-03-08 01:40:22--  https://raw.githubusercontent.com/opencv/opencv/4.x/data/haarcascades/haarcascade_frontalface_default
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 930127 (908K) [text/plain]
Saving to: 'haarcascade_frontalface_default.xml'

haarcascade_frontal 100%[===================>] 908.33K  --.-KB/s    in 0.05s

2026-03-08 01:40:22 (19.1 MB/s) - 'haarcascade_frontalface_default.xml' saved [930127/930127]

(np.float64(-0.5), np.float64(6936.5), np.float64(3406.5), np.float64(-0.5))
```

Detección de rostros- método 2

```
# Importar bibliotecas
import cv2
import matplotlib.pyplot as plt

# Ruta de la imagen
imagePath = r"/content/imagen1.jpg"

# Leer imagen
img = cv2.imread(imagePath)

# Copias para mostrar resultados
img_original = img.copy()
img_gaussian = img.copy()

# Convertir a escala de grises
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# -------- NUEVO METODO: FILTRO GAUSSIANO --------
gaussian = cv2.GaussianBlur(gray,(5,5),0)

# Cargar clasificador de rostros
```

```
face_classifier = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

# -------- DETECCION EN IMAGEN ORIGINAL --------
faces_original = face_classifier.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(40,40)
)

for (x,y,w,h) in faces_original:
    cv2.rectangle(img_original,(x,y),(x+w,y+h),(0,255,0),3)

# -------- DETECCION EN IMAGEN CON FILTRO GAUSSIANO --------
faces_gaussian = face_classifier.detectMultiScale(
    gaussian,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(40,40)
)

for (x,y,w,h) in faces_gaussian:
    cv2.rectangle(img_gaussian,(x,y),(x+w,y+h),(255,0,0),3)

# Convertir a RGB para matplotlib
img_original = cv2.cvtColor(img_original, cv2.COLOR_BGR2RGB)
img_gaussian = cv2.cvtColor(img_gaussian, cv2.COLOR_BGR2RGB)

# -------- MOSTRAR RESULTADOS --------
plt.figure(figsize=(18,8))

plt.subplot(1,2,1)
plt.title("Detección de rostros - Imagen original")
plt.imshow(img_original)
plt.axis("off")

plt.subplot(1,2,2)
plt.title("Detección de rostros - Imagen con filtro Gaussiano")
plt.imshow(img_gaussian)
plt.axis("off")

plt.show()
```

9.-Escribir sobre la diferencia de los resultados de las dos imagenes

1. Aplicación del operador Sobel sobre una imagen previamente procesada con Canny Cuando aplicamos el operador Sobel a una imagen que ya ha sido procesada con el algoritmo Canny, esencialmente estamos intentando encontrar los gradientes de esos bordes ya optimizados por Canny. El operador Sobel no introduce información estructural adicional. Más bien, tiende a reforzar los gradientes ya presentes en los contornos detectados por Canny. En algunos casos, esta operación puede incluso generar efectos no deseados, tales como: • Engrosamiento de los bordes previamente adelgazados por Canny. • Introducción de artefactos derivados de la amplificación del gradiente. • Pérdida parcial de la limpieza y precisión logradas por el proceso de supresión de no-máximos. En consecuencia, el uso de Sobel después de Canny suele tener un valor limitado desde el punto de vista de la detección de bordes, ya que Canny proporciona por sí mismo una representación optimizada y suficientemente robusta de los contornos.
2. Aplicación del operador Sobel directamente sobre la imagen original Cuando Sobel se aplica directamente sobre la imagen original, permite identificar regiones donde se producen variaciones abruptas de intensidad, lo que constituye una aproximación efectiva para la detección inicial de bordes. Sin embargo, esta técnica presenta ciertas limitaciones: • Los bordes resultantes suelen ser más

gruesos que los obtenidos mediante métodos más avanzados. • Es más sensible al ruido, debido a que no incorpora un proceso de suavizado tan robusto como el utilizado en Canny. • La precisión en la localización de los contornos es menor, especialmente en imágenes con variaciones complejas de iluminación o textura.

Conclusión

Aplicar el operador Sobel sobre la imagen original permite obtener una estimación inicial de los bordes basada en el gradiente, aunque con menor precisión y mayor grosor en los contornos. Por otro lado, aplicar Sobel después del algoritmo Canny no suele aportar información adicional relevante, ya que Canny produce bordes más delgados, limpios y robustos. En la práctica, Canny es preferido cuando se requiere una detección de bordes de alta calidad, mientras que Sobel se utiliza principalmente como herramienta de análisis de gradientes o como etapa preliminar en procesos más complejos.

Tarea 2:

1.-Encontrar/ Crear una base de datos de imágenes (2 clases, mínimo 20 imágenes por clase).

2.-Importar la base de datos y visualizar 5 ejemplos por cada clase.

3.-Opcional (dividir la base de datos en entrenamiento y prueba).

4.-Extraemos características de las imágenes.

5.-Creamos un clasificador de imágenes en base a las características extraídas.

6.-Evaluamos el clasificador (al menos exactitud 70%)

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import fetch_lfw_people
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from skimage.feature import hog
from skimage import exposure

# --- 1. CARGAR BASE DE DATOS (2 clases, min 20 fotos) ---
# Filtramos para obtener personajes con al menos 50 imágenes para asegurar calidad
data = fetch_lfw_people(min_faces_per_person=50, resize=0.4)
# Tomamos solo las primeras 2 clases (nombres)
class_names = data.target_names[:2]
mask = np.isin(data.target, [0, 1])

X = data.images[mask]
y = data.target[mask]

print(f"Clases seleccionadas: {class_names}")
print(f"Total de imágenes: {len(X)}")

# --- 2. VISUALIZAR 5 EJEMPLOS POR CLASE ---
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i in range(2):
    idx = np.where(y == i)[0][:5]
    for j, img_idx in enumerate(idx):
        axes[i, j].imshow(X[img_idx], cmap='gray')
        axes[i, j].set_title(class_names[i])
        axes[i, j].axis('off')
plt.show()

# --- 3. DIVIDIR EN ENTRENAMIENTO Y PRUEBA ---
X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# --- 4. EXTRACCIÓN DE CARACTERÍSTICAS (HOG) ---
def extract_hog_features(img_array):
    features_list = []
    for img in img_array:
        # Extraemos el histograma de gradientes orientados
        fd = hog(img, orientations=8, pixels_per_cell=(8, 8),
                 cells_per_block=(2, 2), visualize=False)
        features_list.append(fd)
    return np.array(features_list)
```

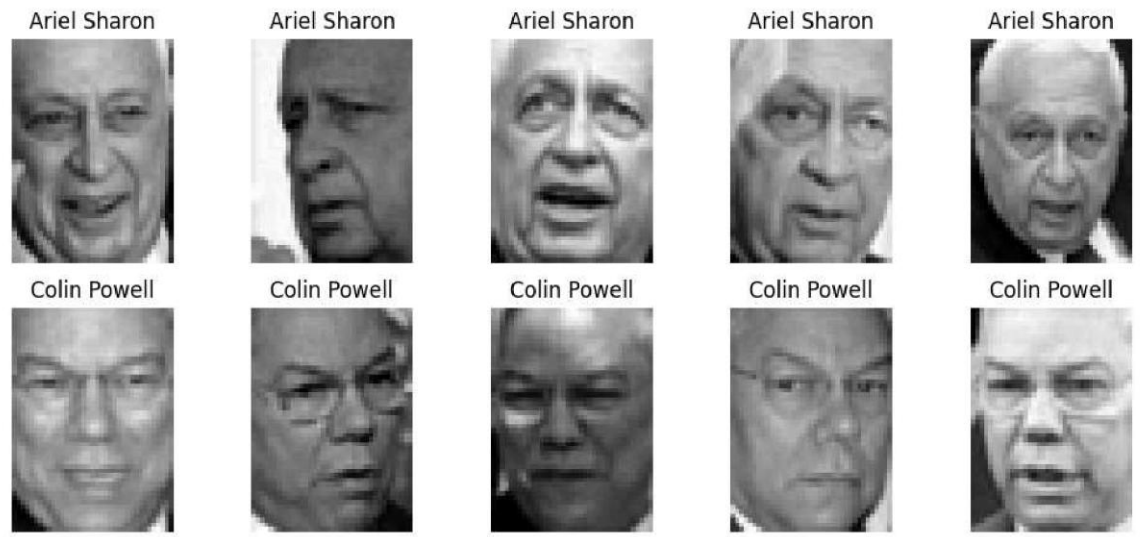
```
X_train = extract_hog_features(X_train_raw)
X_test = extract_hog_features(X_test_raw)

# --- 5. CREAR Y ENTRENAR EL CLASIFICADOR (SVM) ---
clf = SVC(kernel='linear', probability=True, random_state=42)
clf.fit(X_train, y_train)

# --- 6. EVALUACIÓN ---
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(f"\n--- Resultados ---")
print(f"Exactitud (Accuracy): {accuracy * 100:.2f}%")
print("\nReporte de Clasificación:")
print(classification_report(y_test, y_pred, target_names=class_names))
```

```
Clases seleccionadas: ['Ariel Sharon' 'Colin Powell']
Total de imágenes: 313
```




```
--- Resultados ---
Exactitud (Accuracy): 93.62%

Reporte de Clasificación:
              precision    recall  f1-score   support

Ariel Sharon       0.91      0.83      0.87        24
Colin Powell       0.94      0.97      0.96        70

    accuracy                           0.94        94
   macro avg       0.93      0.90      0.91        94
weighted avg       0.94      0.94      0.94        94
```